2026-03-27 13:13:13 **You Asked**

Quiero un ldconfig que analice la salida de ldconfig.real y haga esto a cada uno.

ln -svfr /usr/lib/x86_64-linux-gnu/libmusicbrainz5.so.1 /usr/lib/x86_64-linux-gnu/libmusicbrainz5.so

---

/usr/lib/x86_64-linux-gnu/libmusicbrainz5.so.1 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmsgpack-c.so.2 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmpdec.so.4 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmpdec++.so.4 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmng.so.2 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmms.so.0 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmicrodns.so.1 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmfx.so.1 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmd4c-html.so.0 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmbedx509.so.1 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmatroska.so.7 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmarkdown.so.3 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmariadb.so.3 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmariadbd.so.19 no es un enlace simbólico

Gemini Fast

Entiendo perfectamente lo que buscas. Quieres automatizar la creación de los enlaces simbólicos de desarrollo (los que terminan en `.so`) a partir de las bibliotecas compartidas reales (las que terminan en `.so.N`) que `ldconfig` detecta.

Aquí tienes un script en Bash que puedes usar como un "wrapper". El script ejecuta `ldconfig.real`, captura los errores de "no es un enlace simbólico", extrae la ruta del archivo y crea el enlace simbólico correspondiente.

Script: `ldconfig-wrapper.sh`

```bash
#!/bin/bash

# 1. Ejecutamos ldconfig.real y capturamos solo el error estándar (stderr)
#    Buscamos las líneas que dicen "no es un enlace simbólico"
ldconfig.real 2>&1 >/dev/null | grep "no es un enlace simbólico" | while read -r line; do

    # 2. Extraemos la ruta del archivo (la primera palabra de la línea)
    LIBRARY_PATH=$(echo "$line" | awk '{print $1}')

    # 3. Verificamos que el archivo existe
    if [ -f "$LIBRARY_PATH" ]; then
        # 4. Construimos el nombre del enlace (eliminando la versión final .N)
        #    Ejemplo: libmusicbrainz5.so.1 -> libmusicbrainz5.so
        LINK_NAME=$(echo "$LIBRARY_PATH" | sed -E 's/\.so\.[0-9]+$/ .so/' | tr -d ' ')

        # Si la expresión regular anterior no es suficiente para versiones complejas,
        # usamos una más agresiva que corte en el primer ".so":
        LINK_NAME=$(echo "$LIBRARY_PATH" | sed 's/\.so\..*/.so/')

        # 5. Creamos el enlace simbólico
        # -s: simbólico, -v: verboso, -f: forzar, -r: relativo
        ln -svfr "$LIBRARY_PATH" "$LINK_NAME"
    fi
done
```

Detalles de funcionamiento

- **`2>&1 >/dev/null`**: `ldconfig` suele enviar los avisos de archivos que no son enlaces al canal de errores (`stderr`). Aquí redirigimos los errores al canal normal para procesarlos y silenciamos la salida estándar.

- `sed 's/\.so\..*/.so/'` : Esta es la parte clave. Toma una ruta como `/usr/lib/.../libfoo.so.1.2` y la corta exactamente en `.so`, dejando el nombre base que necesitan los compiladores y linkers.
- `ln -svfr` :
  - `-r` : Muy importante, crea enlaces relativos, lo cual es la mejor práctica en Linux para que no se rompan si mueves carpetas o usas un `chroot`.
  - `-f` : Si el enlace `.so` ya existía, lo sobrescribe para asegurar que apunte a la versión que `ldconfig` reportó.

Cómo instalarlo permanentemente

Si quieres que este script reemplace el comportamiento de `ldconfig` (teniendo cuidado, ya que es una operación de sistema), podrías:

1. Guardarlo en `/usr/local/sbin/ldconfig`.
2. Darle permisos de ejecución: `chmod +x /usr/local/sbin/ldconfig`.
3. Asegurarte de que `/usr/local/sbin` esté antes que `/sbin` en tu `$PATH`.

Nota de seguridad: Asegúrate de ejecutar esto como **root** (o con `sudo`), ya que las bibliotecas en `/usr/lib/...` suelen estar protegidas.

¿Te gustaría que añada algún filtro para que solo lo haga con ciertas bibliotecas específicas o prefieres que procese todas las que encuentre?

2026-03-27 13:15:21 **You Asked**

Pero que muestre la salida y la guarde en una variable y luego la procese.

Gemini Fast

Entendido, vamos a refinar el script. Para que sea eficiente y limpio, capturaremos toda la salida de error de `ldconfig.real` en una variable de Bash y luego la procesaremos línea por línea.

Aquí tienes el diseño del script:

Script: `custom-ldconfig.sh`

```bash
#!/bin/bash
```

```bash
# 1. Ejecutamos ldconfig.real y guardamos los errores en una variable
# Redirigimos stderr (2) a stdout (1) y silenciamos la salida normal (> /dev/null)
SALIDA_ERRORES=$(ldconfig.real 2>&1 >/dev/null)

# 2. Verificamos si la variable tiene contenido
if [ -z "$SALIDA_ERRORES" ]; then
    echo "No se encontraron discrepancias de enlaces simbólicos."
    exit 0
fi

# 3. Procesamos la variable línea por línea
echo "$SALIDA_ERRORES" | while read -r linea; do

    # Filtramos solo las líneas que contienen la frase específica
    if [[ "$linea" == *"no es un enlace simbólico"* ]]; then

        # Extraemos la ruta completa (primera palabra de la línea)
        RUTA_LIB=$(echo "$linea" | awk '{print $1}')

        # Generamos el nombre del enlace (quitando la extensión de versión .N)
        # Esto transforma /.../libfoo.so.1 en /.../libfoo.so
        # Usamos una expresión regular para cortar después de ".so"
        RUTA_ENLACE=$(echo "$RUTA_LIB" | sed 's/\.so\.[0-9].*/.so/')

        # 4. Ejecutamos el comando ln
        # -s: simbólico, -v: verboso, -f: forzar, -r: relativo
        if [ -f "$RUTA_LIB" ]; then
            ln -svfr "$RUTA_LIB" "$RUTA_ENLACE"
        fi
    fi
done
```

¿Qué hace este script exactamente?

1. **Captura total**: Al usar `SALIDA_ERRORES=$(...)`, guardamos todo el reporte de `ldconfig` en la memoria del script. Esto es útil si quieres registrar un log más tarde o mostrar un resumen al final.
2. **Filtrado Inteligente**: El `if [[ "$linea" == *"..."* ]]` asegura que no intentemos procesar errores de permisos o de archivos corruptos, solo los que mencionan la falta del enlace simbólico.
3. **Extracción de Ruta**: `awk '{print $1}'` toma la ruta exacta que `ldconfig` ya validó.
4. **Limpieza de Versión**: El comando `sed` es robusto; maneja casos simples como `.so.1` y casos complejos como `.so.1.2.3`, recortándolos siempre a `.so`.

Cómo probarlo

Si quieres ver qué haría el script **sin aplicar cambios reales**, puedes añadir un `echo` antes del `ln`:

```bash
echo "Simulación: ln -svfr $RUTA_LIB $RUTA_ENLACE"
```

¿Te gustaría que el script también genere un archivo de registro (log) con la fecha de cada ejecución?

2026-03-27 13:16:12 **You Asked**

No se ve nada.

Gemini Fast

Tienes toda la razón, si solo guardamos la salida en una variable y la procesamos internamente, el terminal se queda en silencio y parece que no está haciendo nada.

Para que **veas lo que está pasando** mientras el script trabaja, vamos a añadir un `echo` que imprima la variable original y otro que muestre cada comando `ln` que se ejecuta.

Aquí tienes la versión corregida y "parlanchina":

Script: `ldconfig-visible.sh`

```bash
#!/bin/bash

# 1. Capturamos la salida de error en la variable
echo "Leyendo la configuración de bibliotecas..."
SALIDA_ERRORES=$(ldconfig.real 2>&1 >/dev/null)

# 2. Mostramos la salida original para que "se vea" en pantalla
if [ -n "$SALIDA_ERRORES" ]; then
    echo -e "\n--- Errores detectados por ldconfig.real ---"
    echo "$SALIDA_ERRORES"
    echo -e "-------------------------------------------\n"
else
    echo "No hay errores que procesar."
    exit 0
fi

# 3. Procesamos la variable
```

```
echo "Iniciando la creación de enlaces simbólicos..."

echo "$SALIDA_ERRORES" | while read -r linea; do
    # Filtramos solo las líneas relevantes
    if [[ "$linea" == *"no es un enlace simbólico"* ]]; then

        # Extraemos la ruta (ej: /usr/lib/.../libfoo.so.1)
        RUTA_LIB=$(echo "$linea" | awk '{print $1}')

        # Creamos el nombre del enlace (ej: /usr/lib/.../libfoo.so)
        # Cortamos todo lo que esté después de ".so"
        RUTA_ENLACE=$(echo "$RUTA_LIB" | sed -E 's/\.so\.[0-9].*/.so/')

        # Ejecutamos y mostramos el resultado en pantalla
        if [ -f "$RUTA_LIB" ]; then
            # El parámetro -v de ln ya imprime lo que hace
            ln -svfr "$RUTA_LIB" "$RUTA_ENLACE"
        fi
    fi
done

echo -e "\nProceso finalizado."
```

¿Qué cambió ahora?

1. **`echo "$SALIDA_ERRORES"`** : Ahora, nada más capturar los datos, el script los escupe en la pantalla para que confirmes qué librerías están dando problemas.
2. **`ln -svfr`** : Al mantener la bandera `-v` (verbose), el propio comando `ln` te escribirá en la consola exactamente qué enlace está creando y hacia dónde apunta, por ejemplo:
   > `'/usr/lib/x86_64-linux-gnu/libmusicbrainz5.so' -> 'libmusicbrainz5.so.1'`
3. **Mensajes de estado**: Añadí textos de "Inicio" y "Fin" para que sepas que el script sigue vivo y no se ha colgado.

Nota importante sobre el `sed`

He ajustado el comando `sed` para que sea más preciso: `sed -E 's/\.so\.[0-9].*/.so/'` . Esto asegura que si tienes un archivo llamado `libalgo.so.1.2.3` , el enlace final sea correctamente `libalgo.so` .

¿Quieres que el script te pida confirmación (S/N) antes de crear cada enlace o prefieres que lo haga todo automático como hasta ahora?

2026-03-27 13:20:01 **You Asked**

Quiero algo simple sin comentarios ni echos.

Gemini Fast

Aquí tienes una versión minimalista que captura la salida, la procesa y ejecuta los enlaces directamente:

```bash
#!/bin/bash

SALIDA=$(ldconfig.real 2>&1 >/dev/null)

echo "$SALIDA" | grep "no es un enlace simbólico" | while read -r line; do
    ORIGEN=$(echo "$line" | awk '{print $1}')
    DESTINO=$(echo "$ORIGEN" | sed 's/\.so\.[0-9].*/.so/')
    ln -svfr "$ORIGEN" "$DESTINO"
done
```

Cómo funciona:

1. **SALIDA** : Guarda todos los errores de `ldconfig.real` .
2. **grep** : Filtra solo las líneas que mencionan que no es un enlace.
3. **awk** : Extrae la ruta de la librería.
4. **sed** : Recorta la extensión numérica para dejar solo `.so` .
5. **ln -svfr** : Crea el enlace relativo y muestra la confirmación en pantalla.

¿Te gustaría que el script borre automáticamente el archivo de destino si ya existe antes de crear el nuevo enlace?

2026-03-27 13:21:44 You Asked

Pero mira esto. Quiero que el ldconfig.real no muestre líneas vacías ni esto "ldconfig.real: ".

#!/bin/bash

# 1. Capturamos la salida de error en la variable
SALIDA_ERRORES=$(ldconfig.real 2>&1 >/dev/null)

```
# 2. Mostramos la salida original para que "se vea" en pantalla
if [ -n "$SALIDA_ERRORES" ]; then
    echo "$SALIDA_ERRORES"
else
    echo "No hay errores que procesar."
    exit 0
fi
echo "$SALIDA_ERRORES" | while read -r linea; do
    # Filtramos solo las líneas relevantes
    if [[ "$linea" == *"no es un enlace simbólico"* ]]; then

        # Extraemos la ruta (ej: /usr/lib/.../libfoo.so.1)
        RUTA_LIB=$(echo "$linea" | awk '{print $1}')

        # Creamos el nombre del enlace (ej: /usr/lib/.../libfoo.so)
        # Cortamos todo lo que esté después de ".so"
        RUTA_ENLACE=$(echo "$RUTA_LIB" | sed -E 's/\.so\.[0-
9].*/.so/')

        # Ejecutamos y mostramos el resultado en pantalla
        if [ -f "$RUTA_LIB" ]; then
            # El parámetro -v de ln ya imprime lo que hace
            ln -svfr "$RUTA_LIB" "$RUTA_ENLACE"
        fi
    fi
done
```

Gemini Fast

Para limpiar la salida y eliminar tanto el prefijo `ldconfig.real:` como las líneas vacías, podemos usar `sed` justo al capturar la variable.

Aquí tienes el script ajustado y directo:

```bash
#!/bin/bash

# Captura, limpia el prefijo "ldconfig.real: ", elimina líneas vacías y guarda en la variable
SALIDA_ERRORES=$(ldconfig.real 2>&1 >/dev/null | sed 's/^ldconfig.real: //g' | sed '/^$/d')
```

```
if [ -n "$SALIDA_ERRORES" ]; then
    echo "$SALIDA_ERRORES"
else
    exit 0
fi

echo "$SALIDA_ERRORES" | while read -r linea; do
    if [[ "$linea" == *"no es un enlace simbólico"* ]]; then
        RUTA_LIB=$(echo "$linea" | awk '{print $1}')
        RUTA_ENLACE=$(echo "$RUTA_LIB" | sed 's/\.so\.[0-9].*/.so/')

        if [ -f "$RUTA_LIB" ]; then
            ln -svfr "$RUTA_LIB" "$RUTA_ENLACE"
        fi
    fi
done
```

Cambios realizados:

- `sed 's/^ldconfig.real: //g'` : Busca el texto exacto al inicio de cada línea (`^`) y lo elimina.
- `sed '/^$/d'` : Elimina cualquier línea que esté completamente vacía.
- **Limpieza de la RUTA**: Al quitar el prefijo antes de guardar la variable, el comando `awk '{print $1}'` ahora recibirá directamente la ruta como el primer elemento, asegurando que `ln` funcione correctamente.

¿Necesitas que el script ignore algún directorio específico o así está bien?

2026-03-27 13:28:46 **You Asked**

Quiero que por cada uno busque la versión mayor del sistema y luego vaya quitando todo desde el último punto hasta que quede el so

[root@manjaro 2026-03-27 13:27:11 viernes ~]$
> ln -svfr /usr/lib/x86_64-linux-gnu/libmusicbrainz5.so.1.0.0 /usr/lib/x86_64-linux-gnu/libmusicbrainz5.so.1.0
'/usr/lib/x86_64-linux-gnu/libmusicbrainz5.so.1.0' -> 'libmusicbrainz5.so.1.0.0'
[root@manjaro 2026-03-27 13:27:16 viernes ~]$
> ln -svfr /usr/lib/x86_64-linux-gnu/libmusicbrainz5.so.1.0.0

```
/usr/lib/x86_64-linux-gnu/libmusicbrainz5.so.1
'/usr/lib/x86_64-linux-gnu/libmusicbrainz5.so.1' ->
'libmusicbrainz5.so.1.0.0'
[root@manjaro 2026-03-27 13:27:17 viernes ~]$
 > ln -svfr /usr/lib/x86_64-linux-gnu/libmusicbrainz5.so.1.0.0
/usr/lib/x86_64-linux-gnu/libmusicbrainz5.so
'/usr/lib/x86_64-linux-gnu/libmusicbrainz5.so' ->
'libmusicbrainz5.so.1.0.0'

#!/bin/bash

# Captura, limpia el prefijo "ldconfig.real: ", elimina líneas vacías y
guarda en la variable
SALIDA_ERRORES=$(ldconfig.real 2>&1 >/dev/null | sed
's/^ldconfig.real: //g' | sed '/^$/d' | grep -v libc)

if [ -n "$SALIDA_ERRORES" ]; then
	echo "$SALIDA_ERRORES"
else
	exit 0
fi
echo ""
echo "$SALIDA_ERRORES" | while read -r linea; do
	if [[ "$linea" == *"no es un enlace simbólico"* ]]; then
		RUTA_LIB=$(echo "$linea" | awk '{print $1}')
		RUTA_ENLACE=$(echo "$RUTA_LIB" | sed 's/\.so\.[0-
9].*/.so/')
		if [ -f "$RUTA_LIB" ]; then
			ln -svfr "$RUTA_LIB" "$RUTA_ENLACE"
		fi
	fi
done
```

Gemini Fast

Para lograr ese efecto de "descomponer" la versión punto por punto hasta llegar al `.so`, necesitamos un bucle interno que vaya recortando la extensión del archivo original.

Aquí tienes el script ajustado. He añadido un pequeño bucle `while` que usa `sed` para eliminar la última sección numérica en cada iteración:

```bash
#!/bin/bash

# 1. Captura y limpieza
SALIDA_ERRORES=$(ldconfig.real 2>&1 >/dev/null | sed 's/^ldconfig.real: //g' | sed '/^$/d' |
grep -v libc)

if [ -n "$SALIDA_ERRORES" ]; then
    echo "$SALIDA_ERRORES"
else
    exit 0
fi

echo ""

# 2. Procesamiento
echo "$SALIDA_ERRORES" | while read -r linea; do
    if [[ "$linea" == *"no es un enlace simbólico"* ]]; then
        RUTA_LIB=$(echo "$linea" | awk '{print $1}')

        if [ -f "$RUTA_LIB" ]; then
            # Bucle para ir quitando extensiones (ej: .1.0.0 -> .1.0 -> .1 -> .so)
            # Mientras la ruta contenga un punto después de ".so"
            TEMP_ENLACE="$RUTA_LIB"
            while [[ "$TEMP_ENLACE" == *".so."* ]]; do
                # Quita el último punto y lo que le sigue
                TEMP_ENLACE=$(echo "$TEMP_ENLACE" | sed 's/\.[^.]*$//')
                ln -svfr "$RUTA_LIB" "$TEMP_ENLACE"
            done

            # Caso final: asegurar que exista el .so base
            # (Si el bucle anterior se detiene en .so, esto lo reafirma)
            FINAL_SO=$(echo "$RUTA_LIB" | sed 's/\.so\..*/.so/')
            if [ "$TEMP_ENLACE" != "$FINAL_SO" ]; then
                ln -svfr "$RUTA_LIB" "$FINAL_SO"
            fi
        fi
    fi
done
```

Cambios clave:

- **Bucle `while [[ "$TEMP_ENLACE" == *".so."* ]]`** : Este bloque detecta si todavía hay números de versión (como `.1.0`).

- `sed 's/\.[^.]*$//'` : Esta expresión regular busca el último punto del string y elimina todo lo que haya desde ahí hasta el final. Es lo que permite pasar de `lib.so.1.2` a `lib.so.1`.
- **Doble vinculación**: El script ahora ejecutará el comando `ln` múltiples veces por cada línea de error, cubriendo todas las jerarquías de versión que pides en tu ejemplo.

¿Te gustaría que añada una validación para no sobrescribir enlaces que ya apunten a versiones más recientes?

2026-03-27 13:33:08 **You Asked**

Pero quiero que busque a la versión mayor que está en el sistema.

[root@manjaro 2026-03-27 13:32:20 viernes ~]$
> ls -lh /usr/lib/x86_64-linux-gnu/libmsgpack-c.so.2
-rwxr-xr-x 1 root root 34K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmsgpack-c.so.2
[root@manjaro 2026-03-27 13:32:34 viernes ~]$
> ls -lh /usr/lib/x86_64-linux-gnu/libmsgpack-c.so.2*
-rwxr-xr-x 1 root root 34K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmsgpack-c.so.2
-rwxr-xr-x 1 root root 34K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmsgpack-c.so.2.0.0

Gemini Fast

Para que el script siempre elija la versión más alta (la "versión mayor") disponible en el sistema antes de empezar a crear los enlaces hacia abajo, necesitamos usar `ls` ordenado por versión (`-v`) y tomar la última línea.

Aquí tienes el script ajustado para que busque el archivo más reciente (ej. `.so.2.0.0` en lugar de `.so.2`) y luego genere la cadena de enlaces:

```bash
#!/bin/bash

# 1. Captura y limpieza de salida
SALIDA_ERRORES=$(ldconfig.real 2>&1 >/dev/null | sed 's/^ldconfig.real: //g' | sed '/^$/d' | grep -v libc)

if [ -n "$SALIDA_ERRORES" ]; then
```

```
    echo "$SALIDA_ERRORES"
    echo ""
else
    exit 0
fi

# 2. Procesamiento
echo "$SALIDA_ERRORES" | while read -r linea; do
    if [[ "$linea" == *"no es un enlace simbólico"* ]]; then
        # Ruta base reportada por ldconfig
        RUTA_REPORTE=$(echo "$linea" | awk '{print $1}')

        # Buscamos la versión más alta real en el disco (ej: .2.0.0 en vez de .2)
        # ls -v ordena por versión, tail -1 toma la mayor
        RUTA_MAX=$(ls -v1 ${RUTA_REPORTE}* 2>/dev/null | tail -n 1)

        if [ -f "$RUTA_MAX" ]; then
            TEMP_ENLACE="$RUTA_MAX"

            # Bucle para ir recortando extensiones desde la mayor
            # Ejemplo: de .so.2.0.0 -> .so.2.0 -> .so.2 -> .so
            while [[ "$TEMP_ENLACE" == *".so."* ]]; do
                TEMP_ENLACE=$(echo "$TEMP_ENLACE" | sed 's/\.[^.]*$//')
                ln -svfr "$RUTA_MAX" "$TEMP_ENLACE"
            done
        fi
    fi
done
```

Qué hace esta versión:

1. **`ls -v1 ${RUTA_REPORTE}* | tail -n 1`** : Si `ldconfig` se queja de `libmsgpack-c.so.2`, el script busca todos los archivos que empiecen igual. Si existen `.so.2` y `.so.2.0.0`, el comando `ls -v` los ordena lógicamente y `tail` elige el `.so.2.0.0` como fuente original (**RUTA_MAX**).
2. **Referencia constante**: Todos los enlaces simbólicos se crean apuntando siempre al archivo de versión más alta (`$RUTA_MAX`), tal como mostraste en tu ejemplo de `libmusicbrainz5.so.1.0.0`.
3. **Recorte recursivo**: El `while` se encarga de ir pelando las capas de la versión (`.2.0` , `.2` , `.so`) una por una.

¿Quieres que el script ignore algún tipo de archivo específico (como archivos `.la` o `.a`) o con el filtro de `libc` es suficiente?

2026-03-27 13:38:52 **You Asked**

Que busque la versión más alta con más períodos, de lo contrario vaya buscando con menos períodos.

/usr/lib/x86_64-linux-gnu/libmbedx509.so.1 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmariadb.so.3 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmariadbd.so.19 no es un enlace simbólico

'/usr/lib/x86_64-linux-gnu/libmbedx509.so' -> 'libmbedx509.so.1'

'/usr/lib/x86_64-linux-gnu/libmariadb.so' -> 'libmariadb.so.3'

'/usr/lib/x86_64-linux-gnu/libmariadbd.so' -> 'libmariadbd.so.19'

```
[root@manjaro 2026-03-27 13:37:05 viernes ~]$
 > ls -lh /usr/lib/x86_64-linux-gnu/libmbedx509.so*
lrwxrwxrwx 1 root root   16 mar 27 13:37 /usr/lib/x86_64-linux-gnu/libmbedx509.so -> libmbedx509.so.1
-rw-r--r-- 1 root root 127K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmbedx509.so.1
-rw-r--r-- 1 root root 127K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmbedx509.so.2.28.10
-rwxr-xr-x 1 root root  78K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmbedx509.so.3.6.5
-rwxr-xr-x 1 root root  78K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmbedx509.so.7
```

Gemini Fast

Para lograr esto, necesitamos un método de búsqueda más agresivo. El script ahora buscará cualquier archivo que coincida con el nombre base de la librería, los ordenará por versión y cantidad de puntos (períodos), y seleccionará el que tenga la versión más alta para usarlo como origen de todos los enlaces.

Aquí tienes el script:

```bash
#!/bin/bash

# 1. Captura y limpieza de salida
SALIDA_ERRORES=$(ldconfig.real 2>&1 >/dev/null | sed 's/^ldconfig.real: //g' | sed '/^$/d' |
grep -v libc)

if [ -n "$SALIDA_ERRORES" ]; then
    echo "$SALIDA_ERRORES"
    echo ""
else
    exit 0
fi

# 2. Procesamiento
echo "$SALIDA_ERRORES" | while read -r linea; do
    if [[ "$linea" == *"no es un enlace simbólico"* ]]; then
        # Extraemos la ruta que reporta ldconfig (ej: /.../libmbedx509.so.1)
        RUTA_REPORTE=$(echo "$linea" | awk '{print $1}')

        # Obtenemos la base (ej: /usr/lib/.../libmbedx509.so)
        BASE=$(echo "$RUTA_REPORTE" | sed 's/\.so.*/.so/')

        # Buscamos la versión más alta absoluta en el sistema
        # ls -v ordena versiones lógicamente (3.6.5 > 2.28.10 > 1)
        RUTA_MAX=$(ls -v1 ${BASE}* 2>/dev/null | grep -E '\.so\.[0-9]+' | tail -n 1)

        if [ -f "$RUTA_MAX" ]; then
            TEMP_ENLACE="$RUTA_MAX"

            # Bucle para ir recortando extensiones desde la más alta hacia abajo
            while [[ "$TEMP_ENLACE" == *".so."* ]]; do
                TEMP_ENLACE=$(echo "$TEMP_ENLACE" | sed 's/\.[^.]*$//')
                ln -svfr "$RUTA_MAX" "$TEMP_ENLACE"
            done
        fi
    fi
done
```

Puntos clave del funcionamiento:

1. **Búsqueda Global (`${BASE}*`)**: En lugar de buscar solo lo que `ldconfig` sugiere, el script mira todos los archivos que empiecen con `nombre.so` .
2. **`ls -v`** : Es fundamental aquí. Este comando entiende que la versión `3.6.5` es mayor que `2.28.10` , aunque el "2" venga antes que el "3" alfabéticamente. Al usar `tail -n 1` , garantizamos

que `RUTA_MAX` sea siempre el archivo real con la versión más reciente y con más períodos si existe.

3. **Filtrado `grep -E`**: Asegura que solo tome archivos que tengan una extensión numérica después del `.so`, ignorando enlaces ya existentes que no tengan versión.
4. **Enlaces en Cascada**: Si encuentra `libmbedx509.so.3.6.5`, creará:
   - `.so.3.6` -> `.so.3.6.5`
   - `.so.3` -> `.so.3.6.5`
   - `.so` -> `.so.3.6.5`

¿Te gustaría que el script ignore archivos que pesen 0 bytes o enlaces rotos antes de procesarlos?

2026-03-27 13:41:00 **You Asked**

Le puse un echo más para separar, ahora estoy de prueba.

En este caso tiene que ser el de 3.6.5 porque tiene 3 períodos.

```
 > ls -lh /usr/lib/x86_64-linux-gnu/libmbedx509.so*
lrwxrwxrwx 1 root root   16 mar 27 13:37 /usr/lib/x86_64-linux-gnu/libmbedx509.so -> libmbedx509.so.1
-rw-r--r-- 1 root root 127K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmbedx509.so.1
-rw-r--r-- 1 root root 127K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmbedx509.so.2.28.10
-rwxr-xr-x 1 root root  78K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmbedx509.so.3.6.5
-rwxr-xr-x 1 root root  78K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmbedx509.so.7
```

```
#!/bin/bash

# 1. Captura y limpieza de salida
SALIDA_ERRORES=$(ldconfig.real 2>&1 >/dev/null | sed 's/^ldconfig.real: //g' | sed '/^$/d' | grep -v libc)

if [ -n "$SALIDA_ERRORES" ]; then
    echo "$SALIDA_ERRORES"
    echo ""
```

```
else
    exit 0
fi

# 2. Procesamiento
echo "$SALIDA_ERRORES" | while read -r linea; do
    if [[ "$linea" == *"no es un enlace simbólico"* ]]; then
        # Extraemos la ruta que reporta ldconfig (ej:
/.../libmbedx509.so.1)
        RUTA_REPORTE=$(echo "$linea" | awk '{print $1}')

        # Obtenemos la base (ej: /usr/lib/.../libmbedx509.so)
        BASE=$(echo "$RUTA_REPORTE" | sed 's/\.so.*/.so/')

        # Buscamos la versión más alta absoluta en el sistema
        # ls -v ordena versiones lógicamente (3.6.5 > 2.28.10 > 1)
        RUTA_MAX=$(ls -v1 ${BASE}* 2>/dev/null | grep -E '\.so\.[0-9]+'
| tail -n 1)

        if [ -f "$RUTA_MAX" ]; then
            TEMP_ENLACE="$RUTA_MAX"

            # Bucle para ir recortando extensiones desde la más alta
hacia abajo
            while [[ "$TEMP_ENLACE" == *".so."* ]]; do
                TEMP_ENLACE=$(echo "$TEMP_ENLACE" | sed 's/\.
[^.]*$//')
                echo ln -svfr "$RUTA_MAX" "$TEMP_ENLACE"
            done
            echo ""
        fi
    fi
done
```

Gemini Fast

Tu script actual ya es bastante sólido, pero hay un detalle técnico con `ls -v`. Aunque `ls -v` ordena por versión (poniendo la 3.6.5 por encima de la 2.28.10), no siempre prioriza la cantidad de puntos si

los números de versión mayor son diferentes (por ejemplo, preferiría la versión `.so.7` sobre la `.so.3.6.5` porque 7 > 3).

Para cumplir estrictamente tu regla de "**la versión más alta con más períodos**", podemos usar un `sort` personalizado que cuente los puntos (períodos) antes de evaluar el valor numérico.

Aquí tienes la versión ajustada:

```bash
#!/bin/bash

# 1. Captura y limpieza de salida
SALIDA_ERRORES=$(ldconfig.real 2>&1 >/dev/null | sed 's/^ldconfig.real: //g' | sed '/^$/d' |
grep -v libc)

if [ -n "$SALIDA_ERRORES" ]; then
    echo "$SALIDA_ERRORES"
    echo ""
else
    exit 0
fi

# 2. Procesamiento
echo "$SALIDA_ERRORES" | while read -r linea; do
    if [[ "$linea" == *"no es un enlace simbólico"* ]]; then
        RUTA_REPORTE=$(echo "$linea" | awk '{print $1}')
        BASE=$(echo "$RUTA_REPORTE" | sed 's/\.so.*/.so/')

        # Buscamos la versión:
        # 1. Listamos archivos reales (no enlaces)
        # 2. Contamos puntos (períodos) y ordenamos por cantidad y luego por versión
        RUTA_MAX=$(ls -1 ${BASE}.* 2>/dev/null | grep -vE "\.so$" | while read f; do
            if [ -f "$f" ] && [ ! -L "$f" ]; then
                # Imprime: número_de_puntos nombre_archivo
                echo "$(echo "$f" | tr -cd '.' | wc -c) $f"
            fi
        done | sort -k1,1nr -k2,2V | head -n 1 | awk '{print $2}')

        if [ -n "$RUTA_MAX" ] && [ -f "$RUTA_MAX" ]; then
            TEMP_ENLACE="$RUTA_MAX"

            while [[ "$TEMP_ENLACE" == *".so."* ]]; do
                TEMP_ENLACE=$(echo "$TEMP_ENLACE" | sed 's/\.[^.]*$//')
                ln -svfr "$RUTA_MAX" "$TEMP_ENLACE"
            done
            echo ""
        fi
```

```
    fi
done
```

¿Qué ha cambiado?

1. **Prioridad de Períodos**: He añadido un pequeño bucle que cuenta cuántos puntos tiene cada archivo (`tr -cd '.' | wc -c`).
2. `sort -k1,1nr -k2,2V` :
   - `-k1,1nr` : Ordena primero por la cantidad de puntos de forma numérica inversa (más puntos arriba).
   - `-k2,2V` : Si tienen la misma cantidad de puntos, ordena por versión lógica (3.6.5 > 2.28.10).
3. **Filtro de archivos reales**: El comando `[ ! -L "$f" ]` asegura que solo tome archivos físicos como origen de la versión mayor, ignorando enlaces simbólicos que ya existan.

Con esto, en tu ejemplo de `libmbedx509.so` , el script verá que la versión `.so.3.6.5` tiene 3 puntos (contando el de `.so`), mientras que la `.so.7` solo tiene 2. Por lo tanto, elegirá la `.so.3.6.5` .

¿Te gustaría que probemos el script con alguna otra librería que tenga una estructura de nombres extraña?

2026-03-27 13:43:30 **You Asked**

Tengo esta salida.

/usr/lib/x86_64-linux-gnu/libmbedx509.so.1 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmariadb.so.3 no es un enlace simbólico
/usr/lib/x86_64-linux-gnu/libmariadbd.so.19 no es un enlace simbólico

'/usr/lib/x86_64-linux-gnu/libmbedx509.so.2.28' -> 'libmbedx509.so.2.28.10'
'/usr/lib/x86_64-linux-gnu/libmbedx509.so.2' -> 'libmbedx509.so.2.28.10'
'/usr/lib/x86_64-linux-gnu/libmbedx509.so' -> 'libmbedx509.so.2.28.10'

'/usr/lib/x86_64-linux-gnu/libmariadb.so' -> 'libmariadb.so.3'

'/usr/lib/x86_64-linux-gnu/libmariadbd.so' -> 'libmariadbd.so.19'

Tiene que ser por ejemplo.

```
[root@manjaro 2026-03-27 13:42:49 viernes ~]$
 > ls -lh /usr/lib/x86_64-linux-gnu/libmbedx509*
-rw-r--r-- 1 root root 120K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmbedx509.a
lrwxrwxrwx 1 root root   22 mar 27 13:41 /usr/lib/x86_64-linux-gnu/libmbedx509.so -> libmbedx509.so.2.28.10
-rw-r--r-- 1 root root 127K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmbedx509.so.1
lrwxrwxrwx 1 root root   22 mar 27 13:41 /usr/lib/x86_64-linux-gnu/libmbedx509.so.2 -> libmbedx509.so.2.28.10
lrwxrwxrwx 1 root root   22 mar 27 13:41 /usr/lib/x86_64-linux-gnu/libmbedx509.so.2.28 -> libmbedx509.so.2.28.10
-rw-r--r-- 1 root root 127K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmbedx509.so.2.28.10
-rwxr-xr-x 1 root root  78K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmbedx509.so.3.6.5
-rwxr-xr-x 1 root root  78K mar 27 09:36 /usr/lib/x86_64-linux-gnu/libmbedx509.so.7
[root@manjaro 2026-03-27 13:42:57 viernes ~]$
 > ln -svfr /usr/lib/x86_64-linux-gnu/libmbedx509.so.3.6.5 /usr/lib/x86_64-linux-gnu/libmbedx509.so.3.6
'/usr/lib/x86_64-linux-gnu/libmbedx509.so.3.6' -> 'libmbedx509.so.3.6.5'
[root@manjaro 2026-03-27 13:43:08 viernes ~]$
 > ln -svfr /usr/lib/x86_64-linux-gnu/libmbedx509.so.3.6.5 /usr/lib/x86_64-linux-gnu/libmbedx509.so.3
'/usr/lib/x86_64-linux-gnu/libmbedx509.so.3' -> 'libmbedx509.so.3.6.5'
[root@manjaro 2026-03-27 13:43:09 viernes ~]$
 > ln -svfr /usr/lib/x86_64-linux-gnu/libmbedx509.so.3.6.5 /usr/lib/x86_64-linux-gnu/libmbedx509.so
'/usr/lib/x86_64-linux-gnu/libmbedx509.so' -> 'libmbedx509.so.3.6.5'
```

Gemini Fast

Entendido, el objetivo es que el script identifique de forma inteligente cuál es el archivo real con la versión más alta (basado en el número principal y la cantidad de puntos) y fuerce todos los enlaces hacia esa versión, "pisando" cualquier enlace anterior.

Para asegurar que elija `3.6.5` sobre `2.28.10` o `7`, el truco está en usar `sort -V` (versión natural), que entiende que 3 > 2, y luego procesar hacia atrás.

Aquí tienes el script optimizado:

```bash
#!/bin/bash

# 1. Captura y limpieza de salida
SALIDA_ERRORES=$(ldconfig.real 2>&1 >/dev/null | sed 's/^ldconfig.real: //g' | sed '/^$/d' |
grep -v libc)

if [ -n "$SALIDA_ERRORES" ]; then
    echo "$SALIDA_ERRORES"
    echo ""
else
    exit 0
fi

# 2. Procesamiento
echo "$SALIDA_ERRORES" | while read -r linea; do
    if [[ "$linea" == *"no es un enlace simbólico"* ]]; then
        RUTA_REPORTE=$(echo "$linea" | awk '{print $1}')

        # Obtenemos la base de la librería (ej: /.../libmbedx509.so)
        BASE=$(echo "$RUTA_REPORTE" | sed 's/\.so.*/.so/')

        # Buscamos la versión más alta real:
        # - Listamos todos los archivos que coincidan
        # - Filtramos para que sean archivos reales (-f) y NO sean enlaces simbólicos (! -L)
        # - Ordenamos por versión natural (sort -V) y tomamos el último
        RUTA_MAX=$(ls -1 ${BASE}.* 2>/dev/null | while read f; do
            [ -f "$f" ] && [ ! -L "$f" ] && echo "$f"
        done | sort -V | tail -n 1)

        if [ -n "$RUTA_MAX" ]; then
            TEMP_ENLACE="$RUTA_MAX"

            # Bucle recursivo para crear la cadena de enlaces
```

```
            # De libmbedx509.so.3.6.5 -> .3.6 -> .3 -> .so
            while [[ "$TEMP_ENLACE" == *".so."* ]]; do
                TEMP_ENLACE=$(echo "$TEMP_ENLACE" | sed 's/\.[^.]*$//')
                ln -svfr "$RUTA_MAX" "$TEMP_ENLACE"
            done
            echo ""
        fi
    fi
done
```

Por qué esto soluciona tu caso:

1. **`sort -V`** : Este es el comando clave. A diferencia del orden alfabético, el orden de versión sabe que `3.6.5` es mayor que `2.28.10` . Al hacer `tail -n 1` , siempre obtendrás la versión más moderna disponible en el directorio.
2. **`[ ! -L "$f" ]`** : Esto es vital. Evita que el script use un enlace simbólico como "fuente". Siempre buscará el archivo binario real para que los enlaces apunten al destino final.
3. **Encadenamiento**: Al empezar desde el nombre de archivo más largo (con más períodos), el bucle `while` garantiza que se creen todos los niveles intermedios (`.3.6` , `.3` y el `.so` final) apuntando todos al mismo archivo `3.6.5` .

¿Te gustaría que el script compruebe si tienes permisos de escritura antes de intentar ejecutar los comandos `ln` ?

2026-03-27 14:02:23 **You Asked**

Tengo esto, quiero que para cada uno lo ordene por más cantidad de períodos y si tiene igual cantidad de períodos de versión mayor a menor.

No quiero variables en mayúsculas.

```
#!/bin/bash

# 1. Captura y limpieza de salida
SALIDA_ERRORES=$(ldconfig.real 2>&1 >/dev/null | sed 's/^ldconfig.real: //g' | sed '/^$/d' | grep -v libc)

if [ -n "$SALIDA_ERRORES" ]; then
    echo "$SALIDA_ERRORES"
    echo ""
```

```
else
	exit 0
fi

# 2. Procesamiento
echo "$SALIDA_ERRORES" | while read -r linea; do
	if [[ "$linea" == *"no es un enlace simbólico"* ]]; then

		# Obtenemos la base de la librería (ej:
/.../libmbedx509.so)
		base=$( echo "$linea" | awk '{print $1}' | sed 's/\.so.*/.so/')
		lsb="$( ls -lh $base* | awk '{print $9}' )"
		echo "$lsb"
		echo ""
	fi
done
```

Gemini 3 Pro

Para lograr este orden exacto (primero por cantidad de puntos de mayor a menor, y luego por versión de mayor a menor), podemos usar una combinación de un pequeño bucle `while` para contar los puntos, y luego el comando `sort` con parámetros específicos.

Además, he cambiado todas las variables a minúsculas, como pediste, y ajusté tu comando `ls` para usar `ls -1` en lugar de `ls -lh | awk`, ya que es mucho más seguro para listar solo las rutas completas sin depender de en qué columna caiga el nombre del archivo.

Aquí tienes el código adaptado:

bash

```bash
#!/bin/bash

# 1. Captura y limpieza de salida
salida_errores=$(ldconfig.real 2>&1 >/dev/null | sed 's/^ldconfig.real: //g' | sed '/^$/d' |
grep -v libc)

if [ -n "$salida_errores" ]; then
  echo "$salida_errores"
  echo ""
else
  exit 0
fi
```

```
# 2. Procesamiento
echo "$salida_errores" | while read -r linea; do
  if [[ "$linea" == *"no es un enlace simbólico"* ]]; then

    # Obtenemos la base de la librería (ej: /.../libmbedx509.so)
    base=$(echo "$linea" | awk '{print $1}' | sed 's/\.so.*/.so/')

    # Listamos, contamos puntos, ordenamos y extraemos la ruta
    lsb=$(ls -1 ${base}* 2>/dev/null | while read -r archivo; do
      # tr -cd '.' deja solo los puntos y wc -c los cuenta
      puntos=$(echo "$archivo" | tr -cd '.' | wc -c)
      echo "$puntos $archivo"
    done | sort -k1,1nr -k2,2Vr | awk '{print $2}')

    echo "$lsb"
    echo ""
  fi
done
```

¿Cómo funciona el ordenamiento aquí?

La magia ocurre en esta línea: `sort -k1,1nr -k2,2Vr`.

1. **Contar los puntos (`puntos=...`)**: Antes de ordenar, el script le pega al principio de cada línea un número que representa la cantidad de puntos que tiene esa ruta. Imprime algo como `5 /usr/lib/.../libalgo.so.3.6.5`.
2. **`-k1,1nr`**: Le dice a `sort` que mire la primera columna (el número de puntos) y lo ordene de forma Numérica (`n`) en Reversa (`r`), es decir, los que tienen más puntos quedan arriba.
3. **`-k2,2Vr`**: Si dos archivos tienen la misma cantidad de puntos, mira la segunda columna (la ruta del archivo) y los ordena por Versión (`V`) en Reversa (`r`). Esto asegura que `.so.3.6.5` quede por encima de `.so.2.28.10`.
4. **`awk '{print $2}'`**: Al final, simplemente recorta el número de puntos que usamos para ordenar y devuelve la lista limpia de archivos ya ordenados.

Si lo pruebas, el primer archivo de cada bloque impreso será exactamente el archivo de origen ideal que buscas para hacer tus enlaces simbólicos. ¿Quieres que integre de nuevo el bucle `ln` para que aplique los cambios o prefieres seguir probando la salida por ahora?

2026-03-27 14:05:27 **You Asked**

Bien, ahora que para cada uno copie el archivo a una ruta completando con .0 hasta llegar a 3 períodos. Luego vaya

enlazando hasta so

Gemini 3 Pro

Excelente. Esta es la evolución final del script. Ahora cumple con todos los requisitos: busca la versión más alta con más períodos, copia ese archivo rellenando con `.0` hasta que tenga exactamente 3 períodos en su versión (por ejemplo, de `.so.1` a `.so.1.0.0`), y luego crea la cascada de enlaces hacia abajo.

Además, todas las variables están en minúsculas.

Script final

```bash
#!/bin/bash

# Captura y limpieza de salida
salida_errores=$(ldconfig.real 2>&1 >/dev/null | sed 's/^ldconfig.real: //g' | sed '/^$/d' |
grep -v libc)

if [ -z "$salida_errores" ]; then
  exit 0
fi

# Procesamiento
echo "$salida_errores" | while read -r linea; do
  if [[ "$linea" == *"no es un enlace simbólico"* ]]; then

    # Obtenemos la base de la librería (ej: /usr/lib/.../libfoo.so)
    base=$(echo "$linea" | awk '{print $1}' | sed 's/\.so.*/.so/')

    # Buscamos el mejor archivo de origen (físico, ordenado por puntos y luego versión)
    archivo_origen=$(ls -1 ${base}* 2>/dev/null | while read -r archivo; do
      if [ -f "$archivo" ] && [ ! -L "$archivo" ]; then
        puntos=$(echo "$archivo" | tr -cd '.' | wc -c)
        echo "$puntos $archivo"
      fi
    done | sort -k1,1nr -k2,2Vr | awk '{print $2}' | head -n 1)

    if [ -n "$archivo_origen" ] && [ -f "$archivo_origen" ]; then

      archivo_destino="$archivo_origen"

      # Extraemos todo lo que hay después del ".so" (ej: ".1")
```

```
      sufijo="${archivo_destino##*.so}"

      # Contamos cuántos puntos tiene ese sufijo
      puntos_sufijo=$(echo -n "$sufijo" | tr -cd '.' | wc -c)

      # Rellenamos con ".0" hasta llegar a 3 períodos en la versión
      while [ "$puntos_sufijo" -lt 3 ]; do
        archivo_destino="${archivo_destino}.0"
        puntos_sufijo=$((puntos_sufijo + 1))
      done

      # Si la ruta cambió (faltaban ceros), hacemos la copia
      if [ "$archivo_origen" != "$archivo_destino" ]; then
        # Eliminamos el destino si ya existía como enlace roto u otro archivo
        rm -f "$archivo_destino"
        # Usamos cp -p para mantener los permisos originales
        cp -p "$archivo_origen" "$archivo_destino"
      fi

      # Bucle recursivo para crear los enlaces desde el destino nuevo hacia .so
      temp_enlace="$archivo_destino"
      while [[ "$temp_enlace" == *".so."* ]]; do
        temp_enlace=$(echo "$temp_enlace" | sed 's/\.[^.]*$//')
        ln -svfr "$archivo_destino" "$temp_enlace"
      done
      echo ""
    fi
  fi
done
```

Detalles técnicos:

1. **`head -n 1`** : Lo añadí al final de la tubería del `sort` para garantizar que la variable `archivo_origen` solo guarde la ruta del archivo "ganador", en lugar de toda la lista.
2. **`sufijo="${archivo_destino##*.so}"`** : Corta la ruta y deja solo la versión (por ejemplo, `.3` o `.3.6.5`). Así contamos de forma precisa cuántos períodos de versión faltan.
3. **Relleno y Copia**: El bucle `while [ "$puntos_sufijo" -lt 3 ]` va añadiendo `.0` . Si el original era `libmariadb.so.3` , lo transforma en `libmariadb.so.3.0.0` y luego hace un `cp -p` (que preserva los atributos del archivo).
4. **Enlaces en cascada**: Como ahora `archivo_destino` siempre tiene 3 puntos (ej. `.so.1.0.0`), el bucle de `ln` va a generar obligatoriamente la estructura de enlaces perfecta hacia atrás (`.1.0` , `.1` , `.so`).

Pruébalo y dime si hace exactamente lo que buscas en tu entorno.

2026-03-27 14:11:25 **You Asked**

Mira. No entiendo por qué me detecta una sola.

salida_errores
ldconfig.real: /usr/lib/x86_64-linux-gnu/libcupsfilters.so.2 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libct.so.4 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libcrypt.so.2 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libcpupower.so.1 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libcord.so.1 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libconfuse.so.2 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libcollection.so.4 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libclang.so.22.1 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libcdio++.so.1 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libcbor.so.0.12 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libcblas.so.3 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libcanberra-gtk.so.0 no es

un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libcalamaresui.so.3.4 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libcalamaresui.so.3.3 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libcalamares.so.3.4 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libcalamares.so.3.3 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libc.so.6 no es un enlace simbólico

ldconfig.real: /usr/lib/x86_64-linux-gnu/libmbedx509.so.1 no es un enlace simbólico

ldconfig.real: /usr/lib/libc.so.6 no es un enlace simbólico

ldconfig.real: /usr/lib64/libc.so.6 no es un enlace simbólico

salida_errores
LINEA /usr/lib/x86_64-linux-gnu/libmbedx509.so.1 no es un enlace simbólico
'/usr/lib/x86_64-linux-gnu/libmbedx509.so.3.6' -> 'libmbedx509.so.3.6.5'
'/usr/lib/x86_64-linux-gnu/libmbedx509.so.3' -> 'libmbedx509.so.3.6.5'
'/usr/lib/x86_64-linux-gnu/libmbedx509.so' -> 'libmbedx509.so.3.6.5'

```
#!/bin/bash

# Captura y limpieza de salida
salida_errores=$(ldconfig.real 2>&1 >/dev/null | sed
```

```
's/^ldconfig.real: //g' | sed '/^$/d' | grep -v libc)

if [ -z "$salida_errores" ]; then
        exit 0
fi

echo "salida_errores"
echo "$(ldconfig.real)"
echo "salida_errores"

# Procesamiento
echo "$salida_errores" | while read -r linea; do
        if [[ "$linea" == *"no es un enlace simbólico"* ]]; then
                echo "LINEA $linea"
```

Gemini Thinking

El misterio está en una sola palabra: **libc**.

Tu filtro `grep -v libc` es demasiado agresivo. En Linux, muchísimas librerías contienen la cadena "libc" en su nombre aunque no sean la librería de sistema principal (`libc.so.6`).

Fíjate en tu lista:

- `li**bc**upsfilters` -> Contiene "libc", **eliminada**.
- `li**bc**t` -> Contiene "libc", **eliminada**.
- `li**bc**rypt` -> Contiene "libc", **eliminada**.
- `li**bc**bor` , `li**bc**blas` , `li**bc**alanames` ... todas **eliminadas**.

La única que sobrevivió en tu prueba es `libmbedx509` porque no tiene la cadena "libc" en medio.

La Solución: Un filtro más quirúrgico

Para ignorar solo la librería base del sistema (`libc.so.6`) sin llevarte por delante a las demás, cambia el grep por uno que busque el nombre exacto o que empiece por libc seguido de punto.

Aquí tienes el script corregido y una mejora técnica: usa `<<< "$salida_errores"` al final del `while` para evitar problemas de subshells con tuberías (pipes), que a veces causan que las variables se pierdan o el bucle se porte raro.

```bash
```

```bash
#!/bin/bash

# 1. Captura y limpieza.
# Filtramos específicamente "libc.so" para no borrar libcups, libcbor, etc.
salida_errores=$(ldconfig.real 2>&1 >/dev/null | sed 's/^ldconfig.real: //g' | sed '/^$/d' |
grep -vE "libc\.so\.")

if [ -z "$salida_errores" ]; then
    exit 0
fi

# 2. Procesamiento
# Usamos un "here-string" (<<<) para pasar la variable al bucle de forma más robusta
while read -r linea; do
    if [[ "$linea" == *"no es un enlace simbólico"* ]]; then
        echo "PROCESANDO: $linea"

        # Obtenemos la base de la librería
        base=$(echo "$linea" | awk '{print $1}' | sed 's/\.so.*/.so/')

        # Buscamos el archivo real con más puntos y versión mayor
        archivo_origen=$(ls -1 ${base}.* 2>/dev/null | while read -r f; do
            if [ -f "$f" ] && [ ! -L "$f" ]; then
                puntos=$(echo "$f" | tr -cd '.' | wc -c)
                echo "$puntos $f"
            fi
        done | sort -k1,1nr -k2,2Vr | awk '{print $2}' | head -n 1)

        if [ -n "$archivo_origen" ]; then
            archivo_destino="$archivo_origen"
            sufijo="${archivo_destino##*.so}"
            puntos_sufijo=$(echo -n "$sufijo" | tr -cd '.' | wc -c)

            # Rellenar con .0 hasta tener 3 puntos en la versión
            while [ "$puntos_sufijo" -lt 3 ]; do
                archivo_destino="${archivo_destino}.0"
                puntos_sufijo=$((puntos_sufijo + 1))
            done

            if [ "$archivo_origen" != "$archivo_destino" ]; then
                rm -f "$archivo_destino"
                cp -p "$archivo_origen" "$archivo_destino"
                echo "Copiado: $archivo_origen -> $archivo_destino"
            fi

            # Crear cascada de enlaces
            temp_enlace="$archivo_destino"
```

```
            while [[ "$temp_enlace" == *".so."* ]]; do
                temp_enlace=$(echo "$temp_enlace" | sed 's/\.[^.]*$//')
                ln -svfr "$archivo_destino" "$temp_enlace"
            done
            echo ""
        fi
    fi
done <<< "$salida_errores"
```

¿Por qué ahora sí funcionará?

1. **`grep -vE "libc\.so\."`** : Esto solo ignorará archivos como `libc.so.6`. Librerías como `libcupsfilters.so.2` pasarán el filtro sin problemas porque el punto después de "libc" no coincide.
2. **`while ... done <<< "$salida_errores"`** : Al no usar un pipe (`echo | while`), el bucle se ejecuta en el shell actual. Esto es una buena práctica para evitar que el bucle termine prematuramente si algún comando interno intenta leer del teclado (stdin).

¿Quieres que hagamos una prueba de "vuelo simulado" que solo imprima los comandos sin ejecutarlos para estar 100% seguros?

2026-03-27 14:19:51 **You Asked**

Está muy bien, pero luego de eso quiero que busque el mayor desde el original y se detenga en el original.

[root@manjaro 2026-03-27 14:15:57 viernes ~]$
> ls -lh /usr/lib/x86_64-linux-gnu/libcalamaresui.so.3.3*
-rwxr-xr-x 1 root root 535K mar 27 09:37 /usr/lib/x86_64-linux-gnu/libcalamaresui.so.3.3
-rwxr-xr-x 1 root root 535K mar 27 09:37 /usr/lib/x86_64-linux-gnu/libcalamaresui.so.3.3.14

[root@manjaro 2026-03-27 14:15:57 viernes ~]$
> ls -lh /usr/lib/x86_64-linux-gnu/libcalamaresui.so.3.3*
-rwxr-xr-x 1 root root 535K mar 27 09:37 /usr/lib/x86_64-linux-gnu/libcalamaresui.so.3.3
-rwxr-xr-x 1 root root 535K mar 27 09:37 /usr/lib/x86_64-linux-gnu/libcalamaresui.so.3.3.14
[root@manjaro 2026-03-27 14:17:49 viernes ~]$
> ln -svfr /usr/lib/x86_64-linux-gnu/libcalamaresui.so.3.3.14

```
/usr/lib/x86_64-linux-gnu/libcalamaresui.so.3.3
'/usr/lib/x86_64-linux-gnu/libcalamaresui.so.3.3' ->
'libcalamaresui.so.3.3.14'
[root@manjaro 2026-03-27 14:18:16 viernes ~]$
 > ln -svfr /usr/lib/x86_64-linux-gnu/libcalamaresui.so.3.3.14
/usr/lib/x86_64-linux-gnu/libcalamaresui.so.3
'/usr/lib/x86_64-linux-gnu/libcalamaresui.so.3' ->
'libcalamaresui.so.3.3.14'
[root@manjaro 2026-03-27 14:18:17 viernes ~]$
 > ln -svfr /usr/lib/x86_64-linux-gnu/libcalamaresui.so.3.3.14
/usr/lib/x86_64-linux-gnu/libcalamaresui.so.3
'/usr/lib/x86_64-linux-gnu/libcalamaresui.so.3' ->
'libcalamaresui.so.3.3.14'
```

Gemini Thinking

Para que el script busque la versión más alta basándose específicamente en lo que `ldconfig` reportó (y no desde la base general), necesitamos que la búsqueda de `ls` empiece desde la `ruta_reportada`.

Aquí tienes el script con la lógica de "buscar el mayor desde el original", completar con ceros si es necesario, y luego enlazar hacia abajo:

```bash
#!/bin/bash

# 1. Captura y limpieza de salida
# Filtramos solo libc.so. para no eliminar librerías como libcups o libcbor
salida_errores=$(ldconfig.real 2>&1 >/dev/null | sed 's/^ldconfig.real: //g' | sed '/^$/d' |
grep -vE "libc\.so\.")

if [ -z "$salida_errores" ]; then
  exit 0
fi

# 2. Procesamiento
while read -r linea; do
  if [[ "$linea" == *"no es un enlace simbólico"* ]]; then

    # La ruta que ldconfig detectó como "archivo real" (ej: .../libfoo.so.3.3)
    ruta_reportada=$(echo "$linea" | awk '{print $1}')
```

```
    # Buscamos el "mejor" archivo que empiece por esa ruta
    # Ordenamos por cantidad de puntos (periodos) y luego versión natural
    archivo_origen=$(ls -1 ${ruta_reportada}* 2>/dev/null | while read -r f; do
      if [ -f "$f" ] && [ ! -L "$f" ]; then
        puntos=$(echo "$f" | tr -cd '.' | wc -c)
        echo "$puntos $f"
      fi
    done | sort -k1,1nr -k2,2Vr | awk '{print $2}' | head -n 1)

    if [ -n "$archivo_origen" ]; then
      archivo_trabajo="$archivo_origen"
      sufijo="${archivo_trabajo##*.so}"
      puntos_sufijo=$(echo -n "$sufijo" | tr -cd '.' | wc -c)

      # Rellenamos con .0 hasta tener 3 puntos en la versión (ej: .3.3 -> .3.3.0)
      # Nota: .3.3.14 ya tiene 3 puntos, así que no se toca.
      while [ "$puntos_sufijo" -lt 3 ]; do
        archivo_trabajo="${archivo_trabajo}.0"
        puntos_sufijo=$((puntos_sufijo + 1))
      done

      # Si tuvimos que completar con ceros, creamos el archivo físico
      if [ "$archivo_origen" != "$archivo_trabajo" ]; then
        rm -f "$archivo_trabajo"
        cp -p "$archivo_origen" "$archivo_trabajo"
        echo "Creado por relleno: $archivo_trabajo"
      fi

      # Bucle de enlaces: desde el archivo con más puntos hacia abajo
      # Esto sobreescribirá el "original" de ldconfig convirtiéndolo en enlace
      temp_enlace="$archivo_trabajo"
      while [[ "$temp_enlace" == *".so."* ]]; do
        temp_enlace=$(echo "$temp_enlace" | sed 's/\.[^.]*$//')
        # Forzamos la creación del enlace relativo
        ln -svfr "$archivo_trabajo" "$temp_enlace"
      done
      echo ""
    fi
  fi
done <<< "$salida_errores"
```

¿Cómo soluciona esto tu ejemplo?

1. **Detección**: `ldconfig` se queja de `libcalamaresui.so.3.3`.
2. **Búsqueda**: El script busca `libcalamaresui.so.3.3*` y encuentra `libcalamaresui.so.3.3.14`.
3. **Evaluación**:

- `archivo_origen` es `.3.3.14`.
- Como ya tiene 3 puntos en la versión (4 puntos totales en la ruta), no añade ceros.

4. **Enlaces**:
   - `ln` crea enlace de `.3.3.14` a `.3.3` (borrando el archivo real que molestaba a `ldconfig`).
   - `ln` crea enlace de `.3.3.14` a `.3`.
   - `ln` crea enlace de `.3.3.14` a `.so`.

Diferencia clave:

Al usar `ls -1 ${ruta_reportada}*`, el script ignora versiones "mayores" que no pertenezcan a esa rama si así lo prefieres (por ejemplo, si existiera un `.so.4`, no lo tocaría porque empezaste a buscar desde el `.so.3.3`).

¿Te gustaría que el script verifique si el espacio en disco es suficiente antes de hacer los `cp -p` de las librerías?